

面向可扩展长上下文大语言模型的软硬件协同设计

整数存内计算与 CXL 内存池化

Kaichen Li

理学硕士学位论文

计算机与信息工程系

灵山大学

韩国

2026 年 4 月

摘要

长上下文已经成为大语言模型的重要能力之一，但长上下文解码的主要成本并不来自算术计算本身，而是来自键值缓存（Key-Value cache, KV cache）的存储与搬移。将 KV 缓存扩展到片上高带宽内存（HBM）之外，通常需要借助 Compute Express Link（CXL）进行内存扩容；然而，以主机为中心的 CXL 访问方式会在自回归注意力过程中引入明显的带宽压力和排队时延。

本文围绕这一瓶颈研究了两类互补的体系结构方案。首先，提出 **LKC-CXL-PIM**：一种面向整数计算的 CXL 附加存内计算架构，在内存阵列附近直接执行 KV 缓存注意力。该设计以整数非线性单元（Integer Non-Linear Unit, iNLU）替代浮点非线性数据通路，并通过异常值感知路由保持量化 KV 数据在近内存计算过程中尽量停留在整数域。其次，提出 **DisaggKV**：一种面向多节点服务的分离式 CXL 内存池化框架。DisaggKV 将局部性感知的页面放置策略与 CXL 3.0 端点之间的点对点归约结合起来，使分布式 Softmax 归一化能够通过紧凑标量交换完成，而无需由主机搬运大规模张量。

本文的评估结合了单节点内存行为的 Ramulator 2.0 周期级建模、分布式执行的事件驱动 CXL Fabric 仿真，以及用于逻辑面积和功耗趋势分析的综合代理估计。对于最长达 128K token 的上下文，LK-CXL-PIM 将读取延迟降低了 98.9% 以上，并相对于浮点基线实现了 17% 的整体逻辑层面积缩减。在分布式场景中，DisaggKV 可以在大约 2.9K requests/s 之前维持 50 ms 的尾延迟目标，将面向主机的互联流量降低 90% 以上，并在故障注入实验中实现 95 百分位 416 ms 的恢复延迟。这些结果表明，整数近内存执行与基于 CXL 的点对点归约，为可扩展的长上下文大语言模型服务提供了一条具有实际可行性的体系结构路径。

目录

摘要	i
第一章 引言	1
1.1 长上下文大语言模型的兴起	1
1.2 生成式内存墙	1
1.3 现有基础设施中的挑战	2
1.3.1 CXL 与带宽瓶颈	2
1.3.2 存内计算与格式失配	2
1.4 研究动机与目标	2
1.5 论文贡献	2
1.5.1 LKC-CXL-PIM (单节点体系结构)	2
1.5.2 DisaggKV (多节点服务系统)	3
1.6 论文结构	3
第二章 背景知识	4
2.1 LLM 推理动态与 KV 缓存特征	4
2.2 Compute Express Link (CXL) 与分层内存	4
2.3 排队理论与主机聚合瓶颈	5
2.4 存内计算与模型量化	5
第三章 研究动机：长上下文内存墙	6
3.1 KV 缓存的容量与带宽危机	6
3.2 延迟剖析：时间究竟消耗在哪里	7
3.2.1 PIM 中格式失配的现实问题	8
3.3 小结	8

第四章 LKC-CXL-PIM: 单节点体系结构	9
4.1 系统概览	9
4.2 异常值感知路由与溢出处理	9
4.3 整数非线性单元 (iNLU)	10
第五章 DisaggKV: 多节点服务系统	12
5.1 多租户分离式架构概览	12
5.2 Hypervisor 局部性感知分页	13
5.3 基于点对点 Fabric 的同步	13
5.4 通过异步 FIFO 容忍 Fabric 抖动	14
第六章 实验方法	15
6.1 仿真基础设施	15
6.1.1 内存与 PIM 建模	15
6.1.2 CXL Fabric 拓扑模拟器	15
6.2 工作负载与分析踪迹	16
6.3 故障注入模型	16
第七章 系统评估	17
7.1 单节点性能提升	17
7.2 面积与能效	18
7.3 iNLU 的数学保真度	19
7.4 多节点扩展性与吞吐	21
7.5 网络流量降低与韧性	22
第八章 相关工作	25
8.1 CXL 内存池化与分离	25
8.2 神经网络的存内计算	25
8.3 多租户 LLM 服务系统	26
第九章 结论与未来工作	27
9.1 研究总结	27
9.2 未来展望	27

插图

3.1	上下文长度与 KV 缓存规模	6
3.2	不同上下文长度下的延迟构成	7
4.1	异常值感知数据通路	10
4.2	iNLU 流水线结构	10
5.1	DisaggKV 系统概览	12
5.2	两阶段点对点归约	13
7.1	单节点 decode 加速比	17
7.2	周期级延迟对比	18
7.3	整体面积构成	18
7.4	不同上下文长度下的归一化能耗	19
7.5	iNLU 的 Softmax 逼近精度	19
7.6	异常值缓冲区灵敏度分析	20
7.7	定点 iNLU 下的注意力输出质量	20
7.8	尾延迟与吞吐关系	21
7.9	分布式扩展性	22
7.10	互联流量构成	23
7.11	链路时延敏感性分析	23
7.12	故障恢复延迟对比	24

表格

4.1	非线性单元综合对比	11
6.1	仿真参数	16
7.1	单节点结果汇总	17

第一章 引言

1.1 长上下文大语言模型的兴起

大语言模型 (LLMs) 已经迅速成为研究与生产系统中的核心工作负载。以 LLaMA 和 Qwen 为代表的近期模型表明，模型能力的扩展不仅体现在参数规模上，也体现在有效上下文窗口的持续扩展上，使模型能够处理更长的文档、更大的代码仓库以及持续多轮的对话历史 [1,2]。过去只局限于几千 token 的上下文，如今已经常见于数万甚至数十万 token 的范围，且研究原型仍在继续向更长上下文推进。

这一变化也改变了推理阶段的主导瓶颈。在中等序列长度下，推理常被视为一个稠密线性代数问题；而在长上下文下，系统越来越表现为一种内存密集型流式工作负载，其中 KV 缓存的存储、搬移与同步成为一阶问题。

1.2 生成式内存墙

LLM 推理通常包含两个硬件行为不同的阶段：*prefill* 与 *decode*。Prefill 可以并行处理输入提示词，较好地映射到现代加速器的高吞吐矩阵引擎上；而 decode 则是串行的。每生成一个新 token，当前查询向量都需要与历史所有 token 对应的键和值进行注意力计算，这些历史状态保存在“键值缓存 (KV cache)”中 [3]。

随着序列长度增长，KV 缓存逐渐成为解码阶段跨步保留的主要状态。对于一个 7B 级别模型，在 128,000 token 上下文下，单个请求的 KV 缓存就可能占用数 GB 空间，尚未计入并发用户和批处理开销 [3]。在实际服务系统中，这会很快超出片上高带宽内存 (HBM) 的容量，使长上下文推理从一个纯计算问题转变为同时受制于内存容量和内存带宽的问题。

为扩展容量，系统设计者越来越多地依赖片外内存扩展机制，包括基于 Compute Express Link (CXL) 的内存层级 [4]。但仅仅扩展容量，并不能消除 decode 阶段反复搬运 KV 缓存所带来的带宽和延迟开销。

1.3 现有基础设施中的挑战

1.3.1 CXL 与带宽瓶颈

Compute Express Link 通过将远端内存设备映射进主机地址空间，使内存池化和分层内存扩展成为可能 [5]。这有助于缓解 KV 缓存的容量问题，但也暴露出新的吞吐瓶颈。在传统的主机聚合式设计中，主机或加速器在每一步 decode 时仍需跨互联链路拉取远端 KV 张量。当上下文较长时，这种对稠密张量的反复搬运会迅速占满链路带宽，并在 CXL 交换机端口形成排队，进一步抬高平均延迟与尾延迟 [6]。

1.3.2 存内计算与格式失配

存内计算（PIM）是应对互联饱和的一个有吸引力的方向，因为它将计算移动到 KV 缓存数据附近 [7]。困难在于，现代 LLM 服务栈常常以 INT8 或 INT4 等量化格式保存 KV 缓存，以提高有效内存容量 [8,9]；而许多现有 PIM 设计却默认采用浮点非线性数据通路。二者若直接结合，就必须反复执行反量化、浮点 Softmax 计算和重新量化。由此带来的格式转换开销，会削弱近内存执行原本希望获得的效率优势。

1.4 研究动机与目标

本文的核心动机很直接：长上下文解码的主要限制因素，不再是原始算术吞吐，而是 KV 缓存能否被高效地存储、搬运和同步。如果注意力计算能够在 KV 数据附近完成，而分布式归一化又能通过轻量级归约而不是主机搬运大张量实现，那么单节点和多节点两类瓶颈都有望被同时缓解。

因此，本文有两个目标。第一，设计一种单节点近内存注意力引擎，在保持量化数据通路的同时避免浮点格式转换。第二，将这一思路扩展到分离式 CXL 内存池中，以支持可扩展且对延迟敏感的多租户服务。

1.5 论文贡献

本文的主要贡献分为单节点和分布式两部分：

1.5.1 LKC-CXL-PIM（单节点体系结构）

我们设计了一种面向单节点长上下文解码的 CXL 附加 PIM 体系结构。

- **整数非线性单元 (iNLU):** 提出一种面向整数域的非线性数据通路, 通过范围约简与紧凑多项式流水线逼近 Softmax 中的指数函数, 从而避免在近内存路径中使用浮点 Softmax 模块。
- **异常值感知路由:** 设计双路径执行机制, 使绝大多数激活经过高密度 INT8 阵列处理, 而少量大幅值激活被重定向到小型高精度溢出路径。
- **单节点收益验证:** 基于周期级内存仿真与综合代理估计, 证明该设计在 128K 上下文长度下可将读取延迟降低 98.9% 以上, 并相对于基线实现 17% 的整体逻辑层面积缩减。

1.5.2 DisaggKV (多节点服务系统)

我们进一步将该架构扩展到面向多租户服务的分离式 CXL 内存池。

- **局部性感知放置:** 提出一种区分共享提示页和私有 KV 页的放置策略, 使高复用前缀能够被局部保留, 减少不必要的数据移动。
- **基于 CXL 的点对点归约:** 以端点间交换紧凑的 Global_Max 和 Global_Sum 取代主机聚合, 从而将总互联流量降低约 84%, 并将面向主机的流量降低 90% 以上。
- **尾延迟与恢复能力提升:** 在分布式评估中, DisaggKV 可在大约 2.9K requests/s 之前维持 50 ms 的尾延迟目标, 并在故障注入模型下实现亚秒级恢复。

1.6 论文结构

本文其余内容安排如下。第二章回顾 LLM 推理、CXL 内存系统、排队行为和 PIM 的技术背景。第三章分析提出架构所针对的关键瓶颈。第四章介绍单节点的 LKC-CXL-PIM 设计, 第五章进一步扩展为分布式 DisaggKV 系统。第六章说明仿真与分析方法, 第七章给出对应实验结果。第八章将本文置于相关研究背景下讨论, 第九章总结全文并展望未来工作。

第二章 背景知识

本章回顾理解本文所需的技术背景。首先介绍自回归注意力的结构及由此产生的 KV 缓存规模，然后讨论基于 CXL 的内存扩展、主机聚合系统中的排队行为，以及 PIM 与量化推理之间的相互作用。

2.1 LLM 推理动态与 KV 缓存特征

自回归 Transformer 推理需要在不断增长的历史序列上重复执行自注意力。在 decode 阶段，当前查询向量 Q 必须与前面 token 积累下来的键和值张量进行比较 [10]。标准形式如下：

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d}}\right)V \quad (2.1)$$

对于批大小 B 、序列长度 L 、注意力头数 H 和头维度 d ，FP16 格式 KV 缓存的存储需求为：

$$\text{Size}_{KV} = 2 \times B \times L \times H \times d \times (2 \text{ bytes}) \quad (2.2)$$

由于 KV 缓存随 L 线性增长，长上下文 decode 会越来越偏向内存受限，而非计算受限。已有系统研究表明，长序列生成在时间和能耗上会越来越多地消耗于 KV 缓存管理，而不只是稠密算术本身 [11]。

2.2 Compute Express Link (CXL) 与分层内存

HBM 具有极高带宽，但单个加速器封装内可提供的容量有限。随着模型规模、批大小和上下文长度不断增长，这一固定的片上容量会越来越成为约束。Compute Express Link (CXL) 通过基于 PCIe 的链路支持内存扩展与池化，同时保留 load/store 编程模型，从而缓解这一问题 [4]。

Type-3 CXL 设备可以作为内存扩展器，直接映射到主机内存空间，而无需 RDMA 风格的通信栈 [12]。这种抽象对于长上下文服务很有吸引力，但其性能与本地 HBM 仍存在明显差

距。本地 HBM 访问的延迟通常在几十纳秒量级，聚合带宽可达 TB/s；而 CXL 链路的带宽显著更低，遍历延迟也明显更高 [13]。因此，将 KV 缓存从 HBM 转移到远端 CXL 层级虽然能解决容量问题，却可能进一步加剧 decode 阶段的带宽问题。

2.3 排队理论与主机聚合瓶颈

在当前主机聚合式 CXL 部署中，远端访问仍需经过 Root Complex。当多个端点并发处理长上下文 decode 请求时，面向主机的链路及交换机端口就会成为共享的排队资源。一个常用近似是 M/D/1 队列：

$$W_q = \frac{\rho}{2\mu(1-\rho)} \quad \text{where} \quad \rho = \frac{\lambda}{\mu} \quad (2.3)$$

其中， λ 是聚合到达率， μ 是链路或交换机端口的服务率 [14]。对于长上下文服务， λ 主要由 decode 阶段对大规模 KV 缓存区域的反复拉取驱动。一旦 ρ 接近 1，排队时延便会迅速增长。

这也解释了为什么单纯扩容并不足够。如果 decode 工作负载仍然不断把稠密 KV 张量拉回主机，那么更大的内存池只会把更多流量灌入同一个面向主机的瓶颈。要降低 λ ，就必须改变计算与数据流，而不仅仅是增加内存容量。

2.4 存内计算与模型量化

存内计算将简单算术单元布置在内存阵列附近，使更多数据通路停留在内存设备内部，而不是跨越封装边界 [15]。对于注意力工作负载，这一思路尤其自然，因为键和值本身就存储在内存中。

但在实际系统中，PIM 与 LLM 服务软件的结合会受到量化策略影响。SmoothQuant 和 GPTQ 等方法通过将模型状态和激活映射为低精度整数格式，显著降低了存储占用 [9,16]。相比之下，许多现有 PIM 架构，包括商用 HBM-PIM 设计，往往为了适应更广泛工作负载而配置浮点数据通路 [7]。这就在紧凑存储与高效近内存非线性计算之间形成了张力。

如果量化后的 KV 缓存数据在内存逻辑内部仍需反复上转换为浮点格式，那么格式转换开销就会同时削弱性能与面积效率。因此，本文关注一种面向整数域的替代方案，即直接在整数域中逼近注意力中的非线性部分。

第三章 研究动机：长上下文内存墙

长上下文 LLM 推理改变了服务栈中各部分的权重关系。在短上下文下，稠密矩阵乘法仍是主要开销；而在长上下文下，对 KV 缓存的反复扫描会成为延迟、带宽需求和能耗的主要来源。本章利用代表性踪迹和分析结果，更精确地刻画这一变化。

3.1 KV 缓存的容量与带宽危机

在 decode 阶段，每生成一个 token，系统都要重新访问 prefill 与前续 decode 步骤中积累下来的历史 KV 缓存。因此，所需状态量会直接随上下文长度增长。

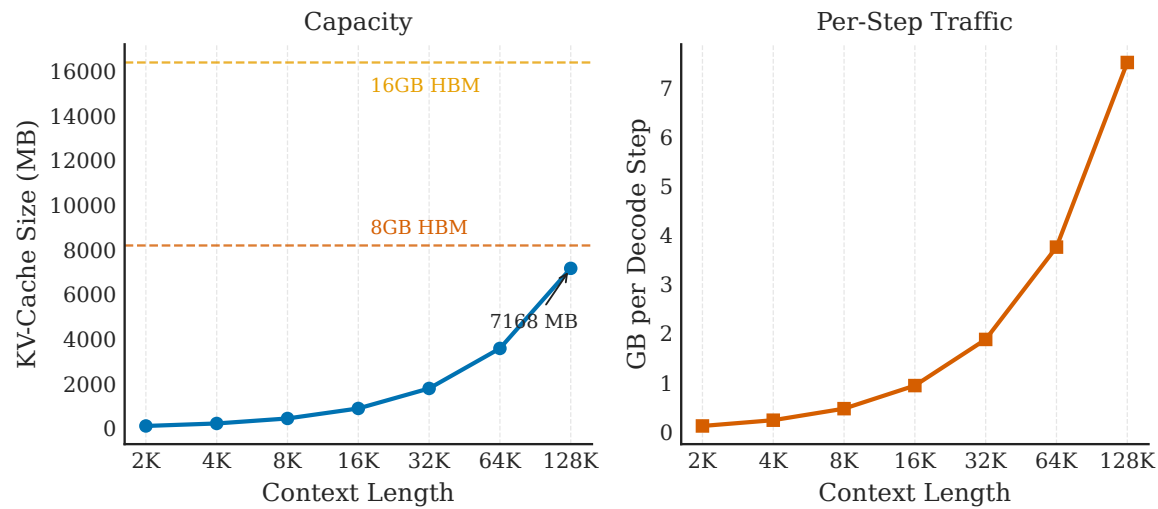


图 3.1: 随着序列长度进入长上下文区间，KV 缓存处理规模迅速增长。

图 3.1 展示了这一增长趋势。以 Qwen2.5-7B 级别模型为例，若采用 28 层、4 个 grouped-query KV 头、每头 128 维以及 FP16 存储，那么单个 128,000 token 请求的 KV 缓存大小将超过 7 GB [3]。只有在系统愿意为单个请求分配大量加速器内存时，这个数字才是可接受的；而在多租户服务场景下，同样的线性增长会直接压缩并发度，并提高内存不足事件发生的概率。

CXL 可以提供所需容量，但流量需求依然严峻。如果系统以 20 tokens/s 的速度生成输出，并不断回取 GB 级历史状态，则所需互联带宽很快就会逼近甚至超过 PCIe/CXL 链路的实际能力 [12]。因此，长上下文服务同时面临内存容量和互联带宽两方面的压力。

3.2 延迟剖析：时间究竟消耗在哪里

为了分析 decode 时间的来源，可以将单个 token 的延迟拆分为三个部分：

1. 矩阵计算：对模型权重和当前 token 状态执行的稠密算术。
2. 反量化开销：在压缩整数表示与浮点计算格式之间进行转换的成本。
3. KV 缓存 I/O 遍历：访问与传输历史键和值的时间。

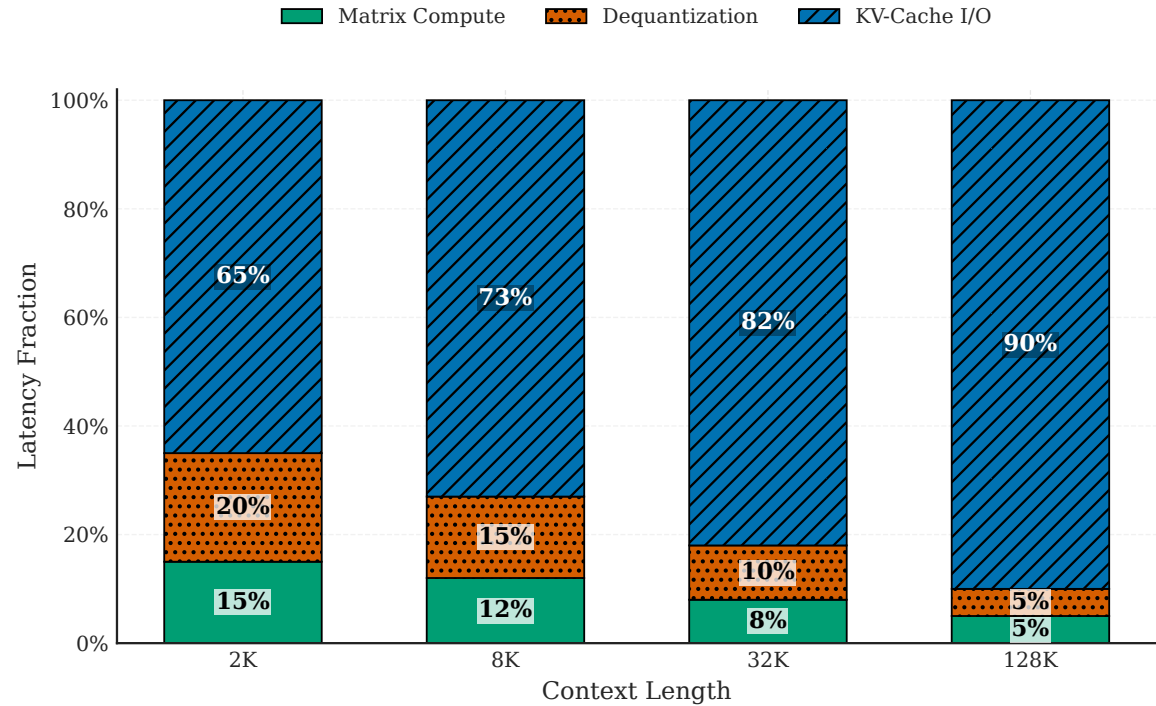


图 3.2: 不同上下文长度下的总体延迟构成。随着序列增长，KV 缓存遍历和格式转换逐渐成为主导项。

图 3.2 表明，主导项会随上下文长度发生变化。在较短上下文下，矩阵计算仍然占据明显比例；但当上下文扩展到 32K 和 128K token 时，KV 缓存搬移已成为主要瓶颈，反量化也仍然是不可忽视的次级开销。这意味着，仅仅继续优化稠密计算单元，并不足以解决端到端 decode 的瓶颈。

3.2.1 PIM 中格式失配的现实问题

上述剖析说明，近内存注意力是一个很有前景的方向，因为它能够避免主机侧反复遍历 KV 缓存。但仍有第二个低效来源存在：量化存储与浮点非线性执行之间的失配。在很多实际部署中，为降低容量压力，KV 缓存张量会采用 INT8 等低精度格式存储 [8]。然而，现有许多近内存计算引擎仍假定采用浮点 Softmax 或其他代价较高的非线性数据通路 [17]。如果没有整数域原生替代方案，系统仍需支付解包、转换、归一化和重新打包的代价。

3.3 小结

本章中的证据表明，一个面向长上下文的可扩展架构至少应满足两个要求。第一，KV 缓存注意力应尽可能在数据附近执行，以避免 decode 阶段持续压满主机互联。第二，注意力中的非线性部分应以能够保持量化数据通路的方式实现，从而避免反复进行浮点格式转换。本文后续章节将分别通过 LKC-CXL-PIM 和 DisaggKV 回应这两个要求。

第四章 LKC-CXL-PIM：单节点体系结构

本章介绍本文的单节点部分，即 **LKC-CXL-PIM** (Long KV Cache via CXL Processing-in-Memory)。该设计针对第三章中识别出的两类单节点瓶颈：一是 KV 缓存在主机互联上的反复搬移，二是量化存储格式与浮点非线性执行之间的反复转换。

4.1 系统概览

LKC-CXL-PIM 将 KV 缓存保存在连接于 CXL 端点的 HBM 堆栈中，作为一层 CXL 附加内存扩展层。在 decode 过程中，主机不再反复把键和值张量拉回本地，而只广播当前查询向量 Q 及相关控制信息。随后，CXL 控制器将该操作分发到 HBM 堆栈的逻辑基底层，在存储 KV 缓存数据附近完成注意力打分和归一化步骤 [18]。

这种重组改变了 decode 阶段的主导数据流。历史稠密张量不再在每一步解码时穿越主机互联，片外接口主要传输的是规模较小的查询向量和响应向量。然而，要让这种数据流真正发挥作用，近内存逻辑就必须在保持整数数据通路的同时执行注意力中的非线性部分。

4.2 异常值感知路由与溢出处理

低精度推理的一大困难在于激活异常值。已有研究表明，大幅值激活虽然出现频率低，但会对量化 Transformer 推理精度造成不成比例的影响 [8]。因此，即便大多数值都处在标称范围内，单一路径的 INT8 数据通路仍会因截断而产生明显误差。

为此，LKC-CXL-PIM 在前端加入了异常值感知逻辑 (**Outlier-Aware Logic**)，用于区分常见激活和少量高幅值激活。

图 4.1 展示了该组织方式。满足 $|x| < \Theta$ 的激活会被量化后送入稠密 INT8 阵列；超出该范围的值则被视为异常值，转入一个小型 INT32 溢出缓冲区。在本文评估的设计中，16-entry 缓冲区已经足以恢复纯 INT8 路径损失的大部分精度，相关结果将在第七章给出。当溢出路径短时拥塞时，控制器会暂停本地流水线，排空缓冲区后再继续执行。由于异常值本身较为稀疏，这一路径很少被触发，对吞吐的影响有限。

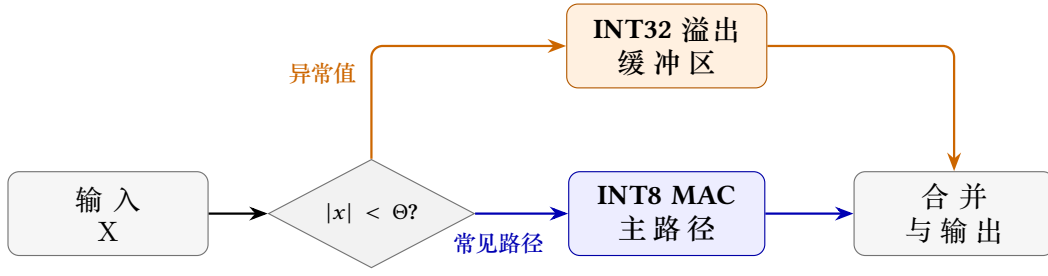


图 4.1: 异常值感知数据通路。绝大多数激活沿 INT8 主路径执行，少量异常值被重定向到高精度溢出缓冲区。

4.3 整数非线性单元 (iNLU)

剩余的挑战在于 Softmax 中的非线性部分。标准实现通常用浮点形式计算指数函数，这既带来面积成本，也引入格式转换开销 [19,20]。LKC-CXL-PIM 为此引入整数非线性单元 (iNLU)，在整数定点域中逼近指数函数。

iNLU 首先利用以 2 为底的分解形式重写指数函数：

$$e^x = 2^{x/\ln 2} \quad (4.1)$$

定义

$$n = \left\lfloor \frac{x}{\ln 2} \right\rfloor, \quad f = x - n \cdot \ln 2 \quad (4.2)$$

则有

$$e^x = 2^n \cdot e^f \quad (\text{其中 } 0 \leq f < \ln 2) \quad (4.3)$$

由于 f 被限制在较小区间内，非线性项 e^f 可以用低阶多项式逼近，而无需大型查找表：

$$e^f \approx A \cdot (f + B)^2 + C \quad (4.4)$$

其中 A 、 B 和 C 为所选定点格式下预先计算好的常数 [21]。

表 4.1 给出了仅针对非线性单元本身的综合对比结果。这些数字只衡量 Softmax 相关逻辑，而不是整个 PIM 流水线的总面积；第七章将进一步讨论整体逻辑层影响。

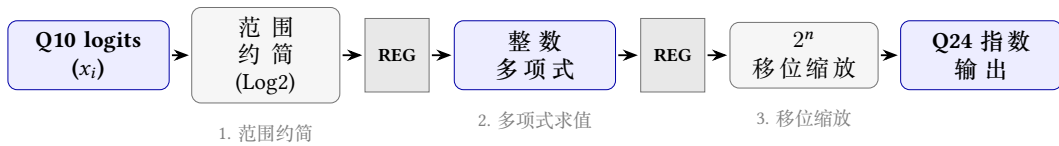


图 4.2: 整数非线性单元 (iNLU) 的流水线组织。

图 4.2 给出了最终数据通路。iNLU 被实现为一个平衡的四级流水线：

表 4.1: 非线性单元本身的硬件综合对比: iNLU 与传统 FP16 Softmax 单元 (TSMC 7 nm, 1 GHz 目标频率)。

指标	FP16 Softmax (基线)	iNLU (本文)	节省
Gate Count (K)	118.4	24.2	79.5%
Die Area (mm ²)	0.0461	0.0094	79.6%
Dynamic Power (mW)	17.8	3.6	79.8%
Max Frequency (GHz)	0.95	1.32	38.9%

1. 范围约简: 计算与 $x/\ln 2$ 对应的整数商和余量。
2. 多项式准备: 构造逼近函数所需的平移残差项。
3. 二次项求值: 在定点域内计算低阶多项式。
4. 归一化: 通过移位操作施加 2^n 缩放。

异常值感知前端与 iNLU 的结合, 使注意力打分和归一化能够在内存阵列附近完成, 而无需引入浮点 Softmax 模块。这正是后续单节点性能和面积收益的关键体系结构基础。

第五章 DisaggKV: 多节点服务系统

单节点 LKC-CXL-PIM 设计降低了单个主机-内存域内部 decode 阶段的数据搬移。但实际服务系统还必须支持大量并发用户，以及超出单个端点可容纳规模的聚合 KV 缓存。本章介绍 **DisaggKV**，这是一个分布式扩展方案，将局部性感知的内存放置与 CXL 附加 PIM 节点之间的点对点归约相结合。

5.1 多租户分离式架构概览

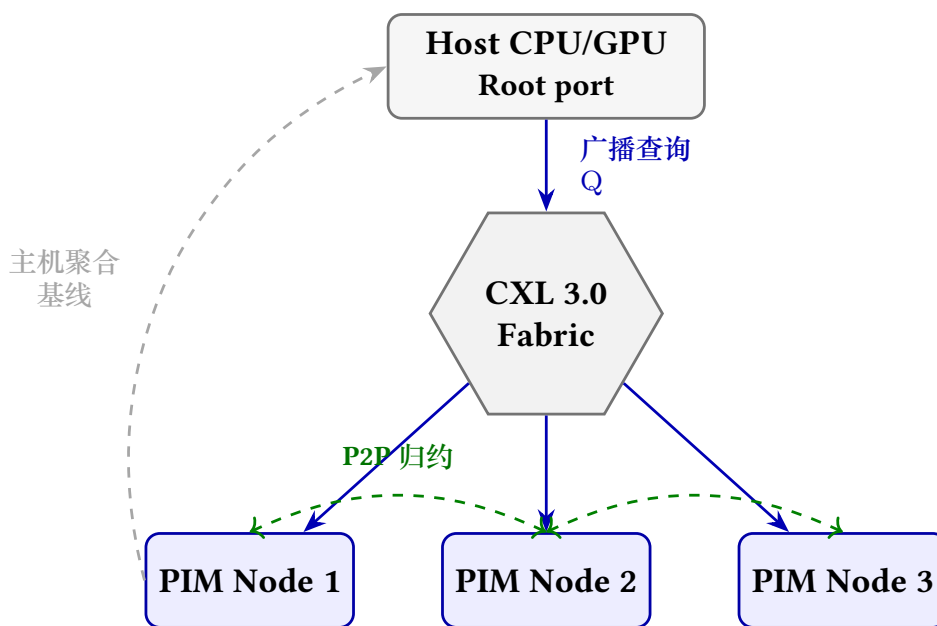


图 5.1: DisaggKV 的体系结构概览。查询向量由主机广播，而分布式 Softmax 屏障所需的同步状态则直接在 CXL 附加 PIM 节点之间交换。

图 5.1 对比了 DisaggKV 与主机聚合基线。在传统设计中，每个远端端点都需要把部分张量或中间结果返回给 Root Complex 进行汇总。对于长上下文 decode，这会在面向主机的链路和交换机端口上形成集中瓶颈 [5]。DisaggKV 则将 CXL Fabric 视为一种归约基底：主机仅发出当前查询并接收最终响应，而分布式 Softmax 所需的中间标量同步状态，则由各 PIM 端点直接交换。

5.2 Hypervisor 局部性感知分页

多租户 LLM 服务同时呈现共享与私有两类内存行为。许多请求会重用同一系统提示词或公共前缀，而用户生成的续写内容和私有历史则保持各自独立。如果放置策略忽略这种差异，就会无谓地复制或迁移共享数据 [22]。

因此，DisaggKV 引入了局部性感知放置策略 (**Locality-Aware Placement Policy**)。共享提示页优先放置在指定的共享节点上，私有 KV 页则分散到其余节点中。其目标不仅是平衡容量，更是减少高复用前缀所导致的跨节点流量。此外，后台迁移守护进程会持续监测节点利用率，并在负载长时间失衡时迁移私有 KV 页。

5.3 基于点对点 Fabric 的同步

分布式注意力不仅需要容量池化，还需要跨节点的一致归一化。各节点本地计算出的 Soft-max 结果不能直接独立组合，因为每个节点都必须基于全局最大值与全局指数和完成归一化 [2]。

DisaggKV 通过两阶段点对点归约协议完成该同步：

1. 屏障阶段 1 (全局最大值)：每个节点先计算本地最大值 $L_{\max}$ ，然后与其他参与节点交换该标量，直到建立共享的 Global_Max。
2. 屏障阶段 2 (全局求和)：每个节点在共享最大值基础上计算本地指数和，并继续参与第二轮归约，生成 Global_Sum。

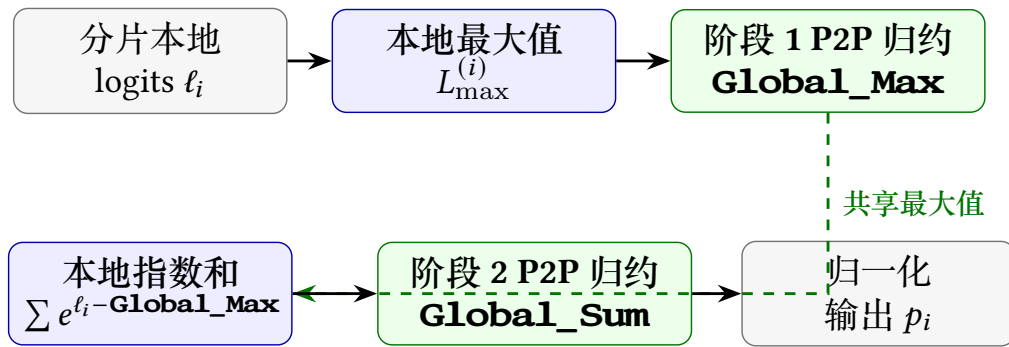


图 5.2: DisaggKV 中的两阶段点对点归约协议。系统先交换全局最大值以保证指数计算的数值稳定性，再交换全局指数和完成最终归一化。

图 5.2 概括了该协议。由于交换的是标量而非大规模张量，通信量得以保持在较小规模。在本文评估配置中，每一步 decode 的同步流量仅为 KB 量级；而若采用主机聚合实现，则需要把较大的 KV 缓存切片重新搬运回主机方向。

5.4 通过异步 FIFO 容忍 Fabric 抖动

基于交换式 Fabric 的分布式执行必须容忍可变延迟。否则，数据包时延变化、瞬时拥塞和时钟域不匹配都可能破坏一个紧耦合的多节点流水线 [23]。为此，DisaggKV 在端点边界引入异步 FIFO。

这些 FIFO 将本地 PIM 流水线与 Fabric 短时抖动解耦。基于 Gray code 的跨时钟域逻辑，使节点能够吸收短时的抖动突发，而不会破坏归约状态。当同步延迟出现时，节点会在本地停顿，而不是立刻向主机传播错误。DisaggKV 还在此基础上加入了轻量级 XOR 奇偶恢复机制，使缺失的子 flit 能够被重建，而无需退回到粗粒度的主机级重启流程 [24]。

第六章 实验方法

本章介绍本文使用的评估方法。由于所提出的系统同时涉及内存时序、近内存逻辑、调度行为以及分布式 CXL Fabric 效应，单一模拟器不足以覆盖全部问题。因此，本文采用周期级单节点内存仿真、事件驱动 Fabric 建模以及基于综合结果的逻辑面积与功耗代理估计相结合的方法。

6.1 仿真基础设施

本文的评估建立在定制化的 **Ramulator 2.0** 框架之上，用于刻画单节点内存行为；同时结合一个更高层的 Python 事件模拟器，用于建模分布式 CXL Fabric 行为 [25]。这种组合方式旨在在关键位置保留细粒度内存时序行为，同时又能让本文分析更大规模的多节点配置。

6.1.1 内存与 PIM 建模

单节点内存子系统被建模为工作在 2 GHz 内存时钟下的 HBM3。Ramulator 2.0 被扩展以支持代表近内存流水线关键阶段的自定义 PIM 指令，包括量化计算和异常值处理。iNLU 数据通路通过定点 golden model 进行检查：输入 logit 采用 Q10 格式，多项式指数路径使用整数运算，归一化输出采用 Q24 整数商；异常值路由逻辑则通过 SystemVerilog 实现进行验证。相关面积与功耗数据以综合代理输入的形式纳入分析，而不被表述为一个完整布局布线芯片的实测结果。

6.1.2 CXL Fabric 拓扑模拟器

多 PIM 节点之间的分布式执行通过一个包裹在单节点时序模型外的事件驱动 CXL Fabric 模拟器进行评估。表 6.1 总结了本文使用的主要系统参数。

该分布式模拟器支持 star 和 fat-tree 拓扑、显式交换机端口服务上限，以及共享链路上的排队时延。本文报告的 artifact 使用当前 CXL 配置文件中的 star 拓扑。这些机制被用来在相同名义带宽和基础时延假设下，对比主机聚合基线与点对点 DisaggKV 协议。

表 6.1: 内存模型与 CXL Fabric 模型所使用的系统仿真参数。

类别	参数	取值
Memory (HBM3)	Base Clock / Data Rate	2.0 GHz / 4.0 Gbps
	Channel Bandwidth	819 GB/s
	tCAS / tRP / tRCD	14.5 / 14.5 / 14.5 ns
CXL 3.0 Fabric	Link Width / Latency	x16 (Gen6) / 150 ns
	Switch Port BW	64 GB/s (Bi-dir)
	Topology	Star

对于吞吐-尾延迟曲线，模拟器计数会与一个校准后的排队模型结合使用。该模型使用 128K token 下 Ramulator 测得的读取延迟下降、CXL 交换机平均排队延迟，以及 128K 延迟分解中显式记录的 KV-cache I/O 占比。生成的数据文件会同时记录这些假设和派生曲线，以便区分实测计数与解析外推。

6.2 工作负载与分析踪迹

工作负载模型基于 **Qwen2.5-7B-Instruct** 配置，既包含单请求长上下文踪迹，也包含多租户服务踪迹。

- **单节点踪迹：**在 2K、8K、32K、64K 和 128K 上下文长度下评估 decode 踪迹，用于比较主机聚合基线与 LKC-CXL-PIM 的周期级差异。
- **多租户踪迹：**分布式实验使用 10–50 req/s 范围内的泊松到达率，并设置 50 个并发虚拟用户。请求既包含长达 8K token 的共享前缀，也包含用户私有对话历史，使放置策略能够同时观测共享与私有访问模式。

6.3 故障注入模型

为评估系统韧性，分布式模拟器包含一个均值无故障时间为 3600 s 的泊松故障注入器。注入事件包括瞬时链路中断和节点性能退化。恢复时延分别针对 DisaggKV 的奇偶辅助恢复方案，以及两种额外开销更高的替代方案进行测量：主机级 RDMA 备份与 checkpoint-restart 恢复。

综合来看，这一方法是混合式的，但内部保持一致：单节点路径采用周期级内存时序，分布式路径采用事件驱动排队与同步建模，而逻辑级效率趋势则由综合代理估计提供支持。

第七章 系统评估

本章给出本文两部分设计的定量结果。前半部分聚焦单节点 LKC-CXL-PIM，后半部分评估分布式 DisaggKV 系统。表 7.1 总结了全文中使用的主要单节点结果。

表 7.1: 基线与 LKC-CXL-PIM 的单节点定量结果汇总。

上下文长度	指标	基线	LKC-CXL-PIM	改进
8K	Read Latency (M Cycles)	10.77	0.12	89.7×
	Row Misses	80,483	1,195	67.3×
128K	Read Latency (M Cycles)	77.90	0.80	97.4×
	Row Misses	288,535	3,700	78.0×
Overall	Logic Area (Relative)	1.00	0.83	17% Savings

7.1 单节点性能提升

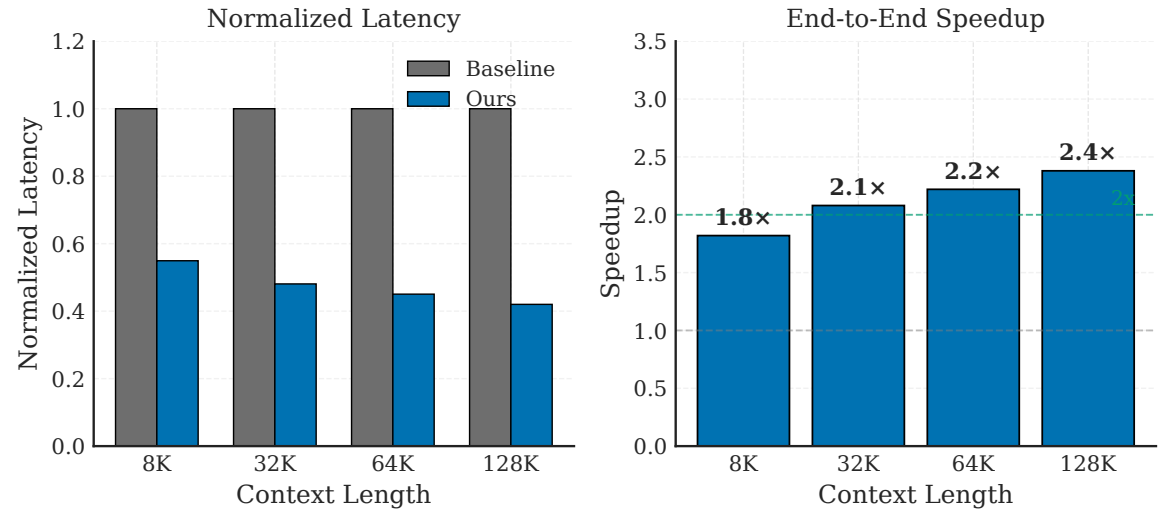


图 7.1: LKC-CXL-PIM 相对于主机聚合基线在不同上下文长度下的端到端 decode 加速比。

图 7.1 给出了端到端 decode 加速比。随着上下文长度增加，近内存执行的优势更加明显，因为基线运行时间中用于搬移 KV 缓存的比例越来越高。测得加速比从 8K 时的 1.8 $\times$ 上升到 128K 时的 2.4 $\times$ ，说明该设计在基线愈发受带宽限制的场景中收益最大。

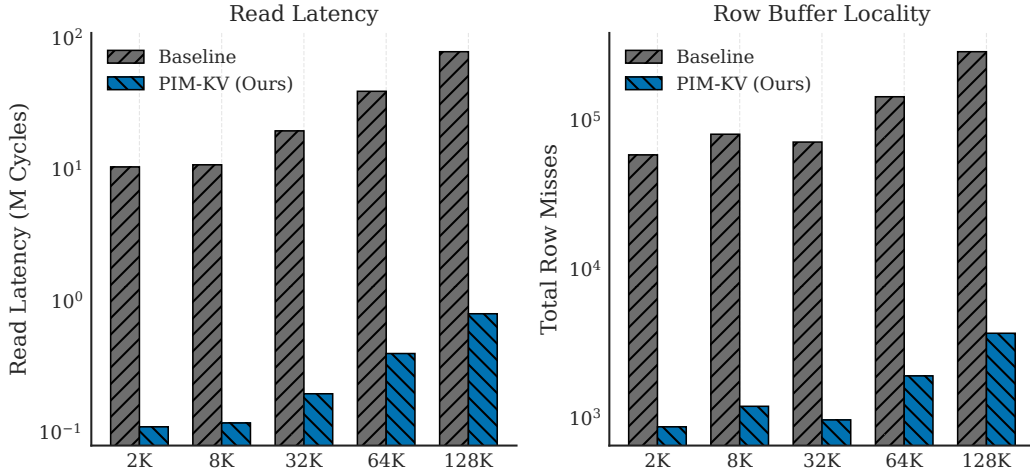


图 7.2: 基线与 LKC-CXL-PIM 在延迟组成和行缓冲局部性方面的周期级对比。

图 7.2 进一步解释了这一加速来源。在 8K 上下文长度下，读取延迟从 10.77 M cycles 降低到 0.12 M cycles；在 128K 下，则从 77.90 M cycles 降低到 0.80 M cycles。相同的数据流重组还改善了行缓冲局部性：row misses 在 8K 时由 80,483 降为 1,195，在 128K 时由 288,535 降为 3,700。这与该设计的目标一致，即尽量将 KV 缓存访问留在内存设备内部，避免主机侧反复拉取。

7.2 面积与能效

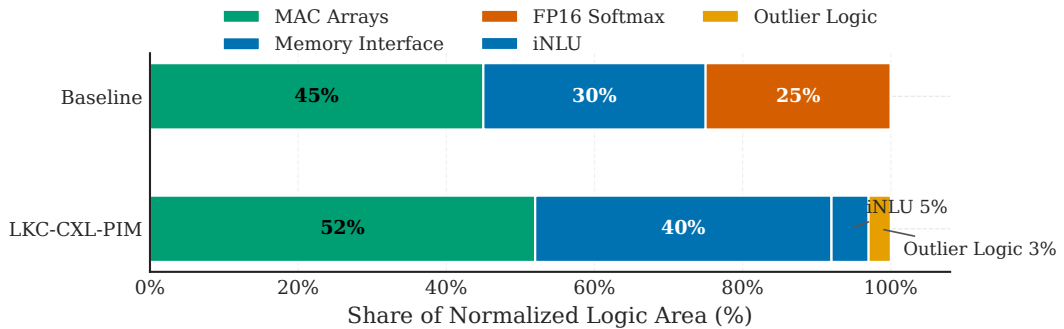


图 7.3: 基线 Softmax 路径与 LKC-CXL-PIM 数据通路的整体逻辑面积构成。

图 7.3 给出了逻辑层级上的面积分解。在基线结构中，FP16 Softmax 模块约占逻辑预算的 25%；而在 LKC-CXL-PIM 中，该模块被 iNLU 与异常值处理路径取代，两者合计约占 8%，从而带来 17% 的整体逻辑面积净缩减。

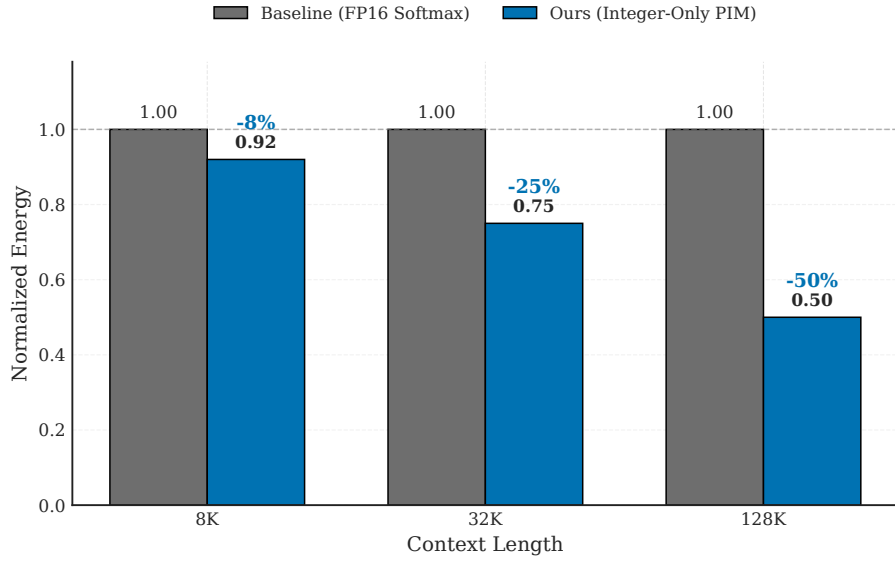


图 7.4: 代表性上下文长度下的归一化能耗对比。

图 7.4 展示了对应的能耗趋势。在 8K、32K 和 128K 上下文长度下，测得的归一化能耗节省分别为 8.3%、24.9% 和约 50.0%。随着上下文长度增加，被避免的片外流量也不断增长，因此节省幅度相应扩大。

7.3 iNLU 的数学保真度

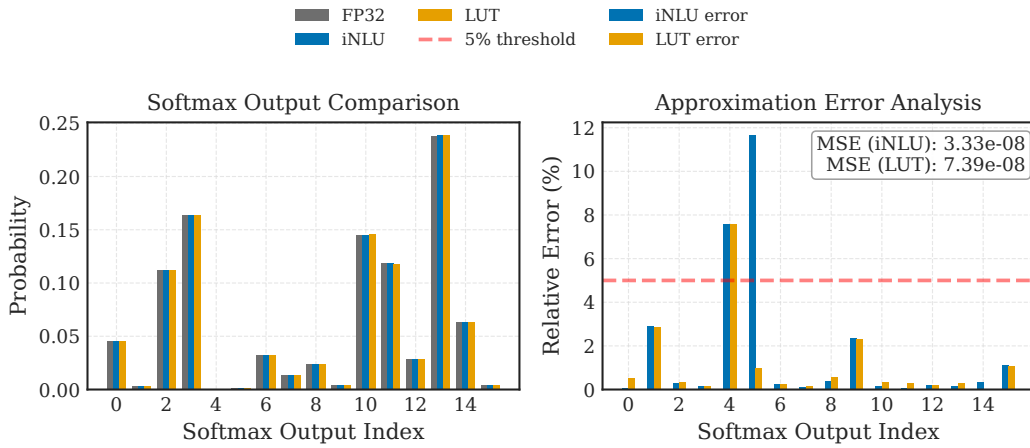


图 7.5: iNLU 多项式路径与查找表近似在 Softmax 输出和逼近误差上的对比。

图 7.5 用于验证整数近似是否足以支持注意力归一化。图中的 iNLU 路径来自定点 golden model: logit 采用 Q10 表示，指数部分使用整数多项式路径，归一化输出采用 Q24 整数商。该路径能够较好地跟踪 FP32 参考结果，并且其均方误差低于图中给出的 64-entry 查找表替代方

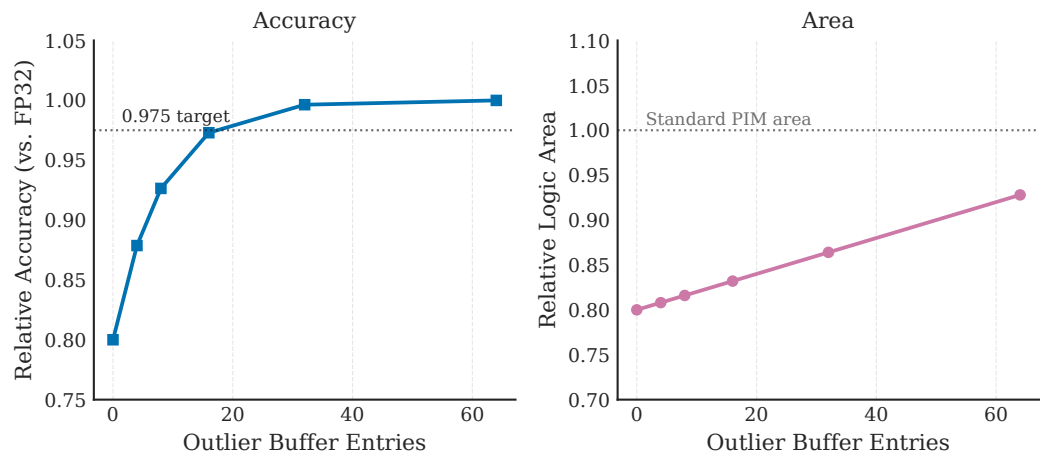


图 7.6: 异常值缓冲区大小对相对精度与相对面积的影响。

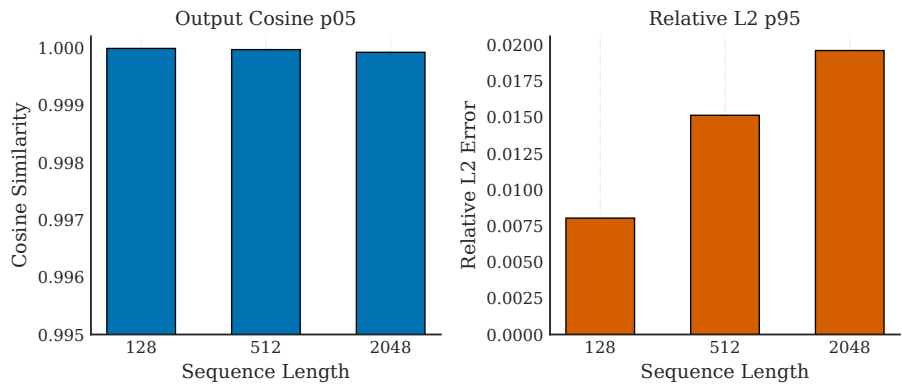


图 7.7: 不同序列长度下定点 iNLU 路径的注意力输出质量。

案（约为 3.33×10^{-8} ，而查找表为 7.39×10^{-8} ）。这说明 iNLU 的硬件缩减并非通过粗糙近似换取。

图 7.7 进一步将 Softmax 层面的误差与注意力行为联系起来。本文使用同一定点路径在一个轻量级单头 Q/K/V attention benchmark 中进行评估，序列长度覆盖 128、512 和 2048 tokens。每个序列长度下进行 128 次随机试验后，输出向量余弦相似度的最差 5 百分位仍高于 0.99992，而输出相对 ℓ_2 误差的最差 95 百分位为 1.96%。这一实验不能替代完整语言模型 perplexity 评估，但说明整数归一化误差在 attention 输出层面仍保持较小。

图 7.6 则评估了溢出路径的作用。纯 INT8 路径的相对精度起点约为 0.80，而 16-entry 缓冲区可将结果提升到约 0.97，同时相对面积大致仍与标准 PIM 基线相当。继续增加到 32 和 64 entries 可以进一步逼近完整精度，但 16 entries 之后的边际收益已明显小于相应面积开销，因此本文将 16-entry 缓冲区视为较为实际的工作点。

7.4 多节点扩展性与吞吐

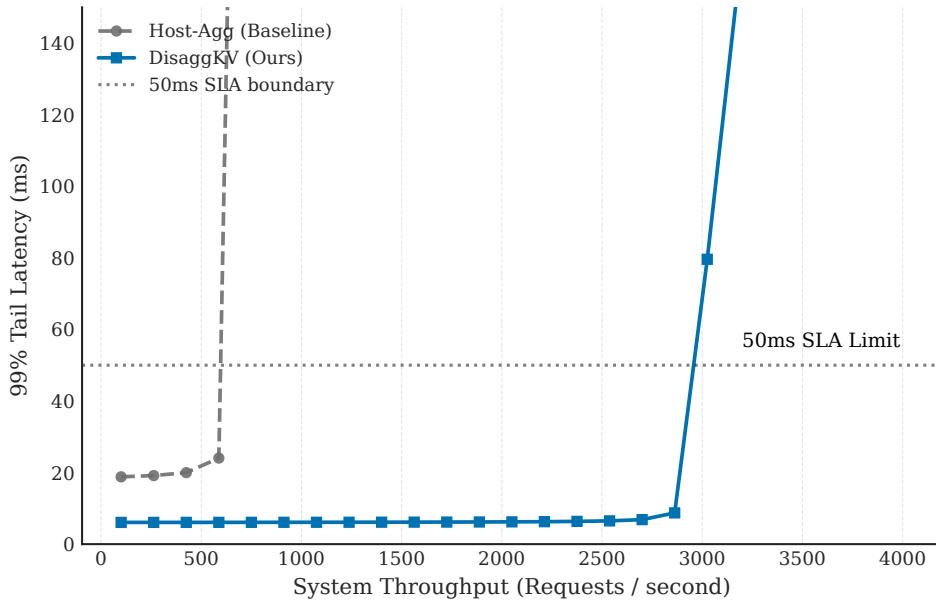


图 7.8: 主机聚合基线与 DisaggKV 的尾延迟-吞吐关系。

图 7.8 基于第六章所述的校准排队模型，比较吞吐与 99 百分位尾延迟。主机聚合基线在略高于 600 requests/s 时越过 50 ms 的 SLA 边界；而 DisaggKV 在 2.7K requests/s 之前始终保持低于 10 ms，并直到略低于 3.0K requests/s 才越过 50 ms 目标。以 SLA 边界为基准，这对应于相对于基线约 4.5× 的吞吐提升。

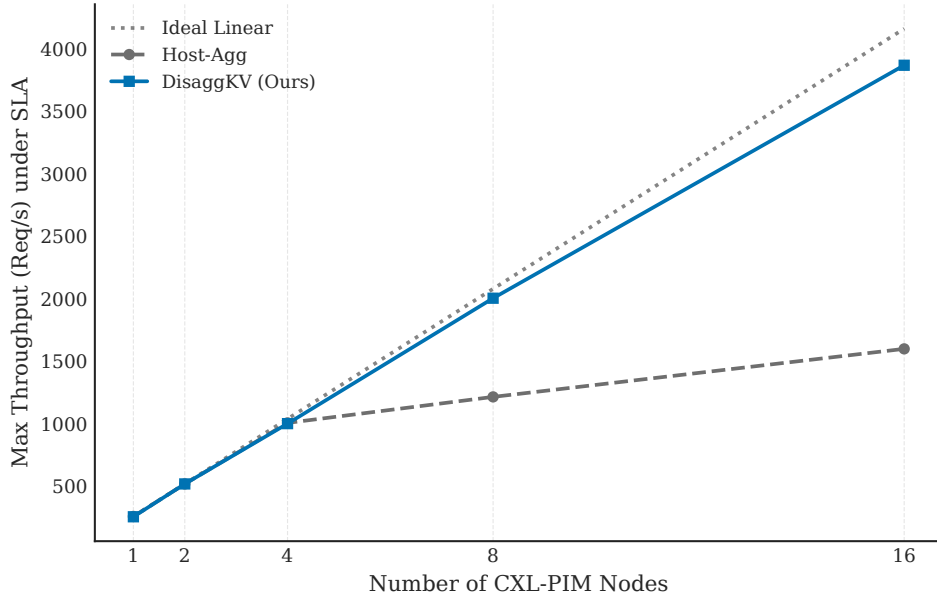


图 7.9: 1 至 16 个节点下的分布式吞吐扩展。

图 7.9 展示了吞吐随 CXL 附加存储节点数的变化。吞吐从 1 个节点时的 257.4 requests/s 增长到 8 个节点时的 2004.9 requests/s，对应 7.79 $\times$ 的提升。在 16 个节点时，吞吐达到 3867.7 requests/s，说明扩展仍在继续，但超过 8 个节点后边际收益开始更加明显地下降。

7.5 网络流量降低与韧性

图 7.10 给出了 DisaggKV 引入的流量权衡。由于更多工作在内存端点内部完成，本地 HBM 访问量有所增加。与此同时，面向主机的 CXL 流量从每 1M tokens 的 3.624 GB 降低到 0.282 GB，降幅达 92.2%。若将点对点归约流量也一并计入，总互联流量仍相对于主机聚合基线下降约 84.4%。这些数值由当前 CXL Fabric artifact 派生，并按照 50-request 踪迹的 token 数进行归一化。这正是 DisaggKV 能在达到 SLA 边界之前维持更高吞吐的核心原因。

图 7.11 评估了系统对 CXL 链路时延增加的敏感性。当链路时延从 100 ns 增加到 1 μ s 时，主机聚合基线的吞吐从约 1200 requests/s 下降到约 230 requests/s；而 DisaggKV 则从约 2780 requests/s 下降到约 1930 requests/s。也就是说，DisaggKV 并非对时延完全不敏感，但由于其通信模式主要由紧凑标量归约而不是稠密张量拉取构成，因此对更长路径显著更具容忍性。

图 7.12 给出了故障注入模型下的韧性结果。基于奇偶辅助的 DisaggKV 恢复路径在故障注入 artifact 中达到 95 百分位 416.384 ms；作为解析对比基线，主机级 RDMA 备份和 checkpoint-restart 恢复分别需要 2.8 s 和 15 s。由此可见，端点驱动的恢复机制不仅改善了故障隔离效果，也显著缩短了恢复时间，并避免了完整的主机侧重启。

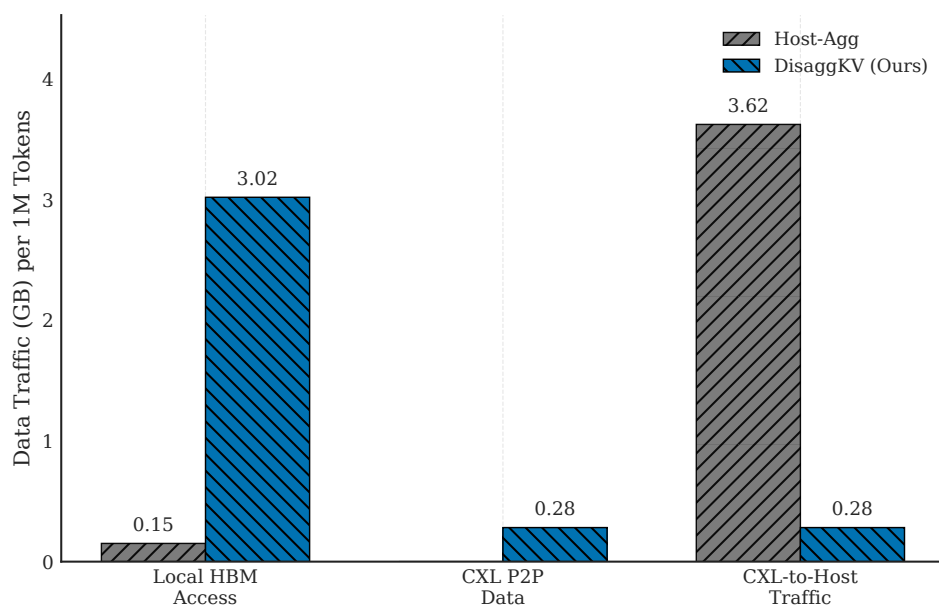


图 7.10: 每 1M decoded tokens 对应的本地 HBM、点对点 and 面向主机流量分解。

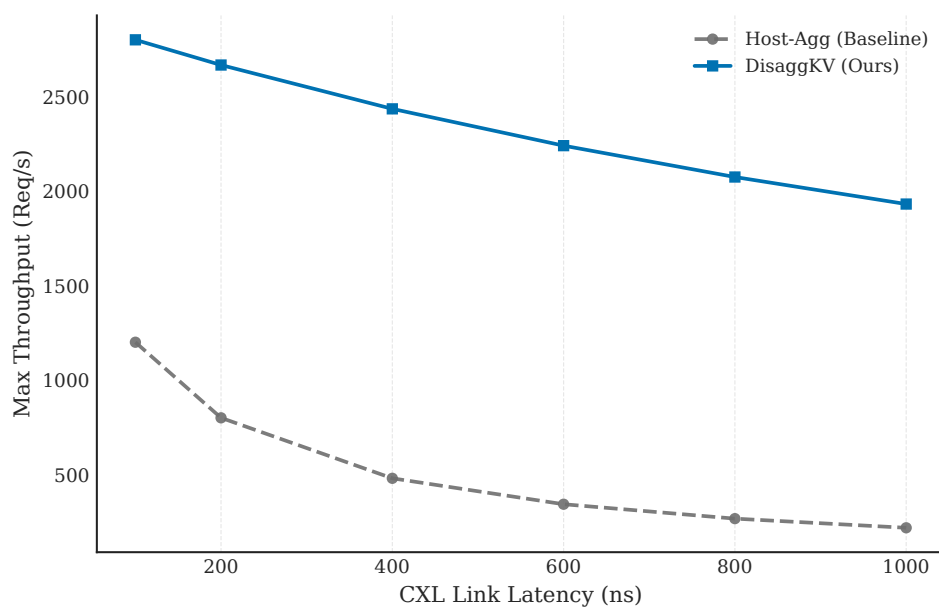


图 7.11: 吞吐对 CXL 链路时延增加的敏感性。

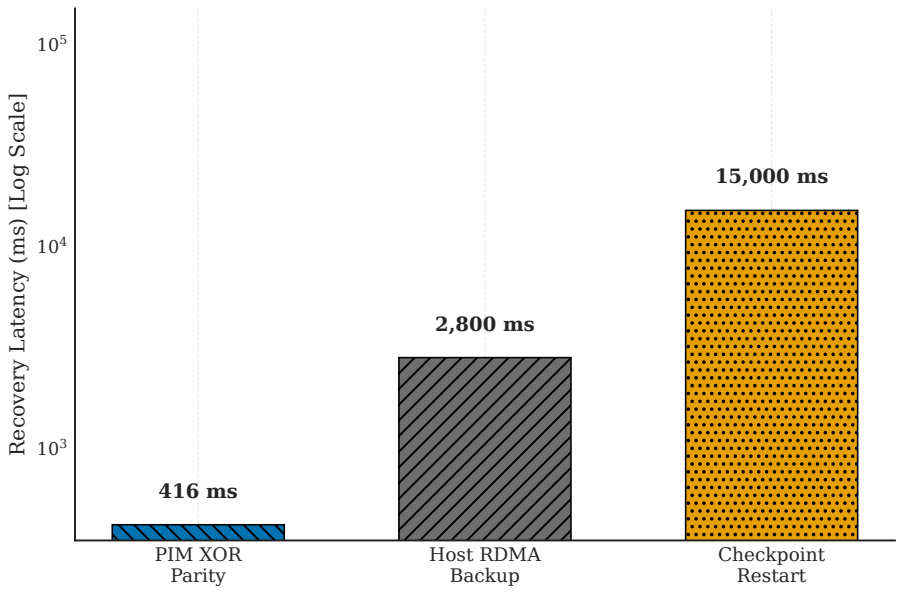


图 7.12: 本文奇偶辅助恢复方案与两种主机管理式恢复替代方案的故障恢复延迟对比。

第八章 相关工作

本章从基于 CXL 的内存分离、近内存加速和 LLM 服务系统三个方向，对本文的工作进行定位。

8.1 CXL 内存池化与分离

近期研究已经将 CXL 确立为一种具有现实可行性的内存扩展与池化基础设施。Pond 研究了云平台中的内存池化，DirectCXL 和 TPP 则探索了 CXL 使能内存层级中的直接访问与透明页放置策略 [5, 12, 13]。类似地，CXL-ANNS 也将相关思想用于大规模、内存密集型数据结构 [26]。

这些系统大多将 CXL 视为一种容量资源，并保留以主机为中心的 load/store 访问模型。本文关注的则是另一个瓶颈：分布式 KV 缓存状态上的 decode 阶段注意力归一化。因此，DisaggKV 不仅做容量池化，还进一步引入了面向长上下文推理的端点驱动点对点归约。

8.2 神经网络的存内计算

现代 PIM 的重新兴起，很大程度上源于机器学习工作负载中数据搬移成本的持续攀升。Samsung 的 HBM-PIM 及相关 accelerator-in-memory 设计，证明了在堆叠内存逻辑层中或其附近集成算术单元的可行性 [7, 27]。其他工作如 TransPIM 和 AttentionLego，则进一步探索了面向 Transformer 的内存中心加速方案 [18, 28]。

然而，大多数既有 PIM 提案要么保留浮点非线性支持，要么面向更广泛的工作负载类别。相比之下，LKC-CXL-PIM 更专注于量化注意力执行，并强调在非线性阶段保持整数导向的数据通路。从这一点上说，它在思想上更接近 I-BERT 这类纯整数定量化工作，但其应用对象不是传统加速器流水线，而是近内存注意力引擎 [17, 21, 29]。

8.3 多租户 LLM 服务系统

系统领域也已经形成了丰富的高效 LLM 服务研究。vLLM 通过 PagedAttention 改进了单节点服务栈中的 KV 缓存管理 [3]。Orca 和 FlexGen 则从调度与内存管理角度研究了高吞吐自回归生成 [30, 31]。DistServe 与 Splitwise 进一步将 prefill 和 decode 分离到不同的计算资源上, 以提升整体服务效率 [6, 22]。

在更大规模场景下, Ring Attention 和 sequence parallelism 通过多设备协同分布式执行注意力计算 [2, 32]。这类方法通常默认使用加速器级链路和主机可见的集体通信机制。DisaggKV 则研究了一种成本更低、面向 CXL 的场景, 其中分布式注意力通过端点侧标量归约和局部性感知页放置来协调完成。

第九章 结论与未来工作

长上下文 LLM 服务越来越受到 KV 缓存存储、搬移与同步成本的约束。本文分别从单节点内存层级和分布式 CXL 内存池两个层面研究了这一问题。

9.1 研究总结

LKC-CXL-PIM 通过将注意力计算移动到存储 KV 缓存数据附近,并在非线性阶段保持整数导向的数据通路,缓解了单节点瓶颈。iNLU 以紧凑的定点近似替代浮点 Softmax 模块,而异常值感知路由路径则为少量大幅值激活保留精度。在本文评估设定下,这一组合在 128K 上下文长度下将读取延迟降低了 98.9% 以上,并实现了 17% 的整体逻辑层面积缩减。

DisaggKV 则将同样的设计思想扩展到分布式系统。通过将局部性感知页面放置与 CXL 附加 PIM 节点之间的点对点标量归约结合起来,它减少了分布式 Softmax 评估中面向主机的互联流量需求。在本文评估的多节点场景中,DisaggKV 可在大约 2.9K requests/s 之前维持 50 ms 的尾延迟目标,在 8 个节点范围内保持接近线性的扩展趋势,并在故障注入实验中实现亚秒级恢复。

9.2 未来展望

本文仍有若干值得继续推进的方向。首先,目前的评估采用了周期级内存仿真、事件驱动 Fabric 仿真和综合代理估计相结合的混合方法;未来可以通过更完整的 RTL 或硬件原型平台,对相同思想进行进一步验证。其次,分布式调度器仍可扩展到更大规模 Fabric、更丰富的拓扑,以及更动态的工作负载组合。第三,若能补充端到端模型质量研究,例如 iNLU 近似下的 perplexity 或服务质量测量,将进一步加强体系结构效率与应用层行为之间的联系。

尽管这些问题仍待深入,本文结果已经表明,长上下文服务并不必然受限于以主机为中心的 KV 缓存搬移。整数近内存执行与基于 CXL 的点对点归约,为长上下文大语言模型系统的扩展提供了一条具有现实意义的体系结构方向。

参考文献

- [1] H. Touvron *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [2] H. Liu, M. Zaharia, and P. Abbeel, “Ring Attention with blockwise transformers for near-infinite context,” in *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [3] W. Kwon *et al.*, “Efficient memory management for large language model serving with Page-dAttention,” in *ACM Symposium on Operating Systems Principles (SOSP)*, 2023.
- [4] CXL Consortium, “Compute Express Link (CXL) Specification Revision 3.0,” 2022.
- [5] H. Li *et al.*, “Pond: CXL-based memory pooling systems for cloud platforms,” in *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.
- [6] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, “DistServe: Disaggregating prefill and decoding for goodput-optimized large language model serving,” in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
- [7] S. Lee *et al.*, “Hardware architecture and software stack for PIM based on commercial DRAM technology,” in *ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*, 2021.
- [8] T. Dettmers *et al.*, “LLM.int8(): 8-bit matrix multiplication for transformers at scale,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- [9] E. Frantar *et al.*, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in *International Conference on Learning Representations (ICLR)*, 2023.
- [10] T. Dao *et al.*, “FlashAttention: Fast and memory-efficient exact attention with IO-awareness,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.

-
- [11] L. Lee *et al.*, “InfiniGen: Efficient generative inference of large language models with dynamic KV cache management,” in *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2024.
 - [12] D. Gouk *et al.*, “Direct access, high-performance memory disaggregation with DirectCXL,” in *USENIX Annual Technical Conference (ATC)*, 2022.
 - [13] H. Al Maruf *et al.*, “TPP: Transparent page placement for CXL-enabled tiered-memory,” in *International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*, 2023.
 - [14] L. Kleinrock, *Queueing Systems, Volume 1: Theory*. Wiley-Interscience, 1975.
 - [15] S. Ghose *et al.*, “Processing-in-Memory: A workload-driven perspective,” *IBM Journal of Research and Development*, vol. 63, no. 6, 2019.
 - [16] G. Xiao *et al.*, “SmoothQuant: Accurate and efficient post-training quantization for large language models,” in *International Conference on Machine Learning (ICML)*, 2023.
 - [17] J. Wang *et al.*, “A 28nm 1.1tops/w integer-only processing-in-memory accelerator for transformer inference,” in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2023.
 - [18] Y. Luo *et al.*, “AttentionLego: An open-source building block for spatially-scalable PIM accelerators,” in *Design Automation Conference (DAC)*, 2024.
 - [19] T. Dao, “FlashAttention-2: Faster attention with better parallelism and workpartitioning,” *arXiv preprint arXiv:2307.08691*, 2023.
 - [20] M. He *et al.*, “Newton: A DRAM-maker’s accelerator-in-memory (AiM) architecture for machine learning,” in *53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, 2020.
 - [21] S. Kim *et al.*, “I-BERT: Integer-only BERT quantization,” in *International Conference on Machine Learning (ICML)*, 2021.
 - [22] P. Patel, E. Choukse, C. Zhang, A. Shah, Í. Goiri, S. Maleki, and R. Bianchini, “Splitwise: Efficient generative LLM inference using phase splitting,” in *Proceedings of the 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024.
 - [23] J. Duato *et al.*, *Interconnection Networks: An Engineering Approach*. Morgan Kaufmann, 2002.

- [24] Y. Li *et al.*, “Understanding CXL memory module failures in large-scale deployments,” *arXiv preprint*, 2024.
- [25] H. Luo *et al.*, “Ramulator 2.0: A modern, modular, and extensible DRAM simulator,” *IEEE Computer Architecture Letters (CAL)*, 2023.
- [26] J. Sun *et al.*, “CXL-ANNS: Software-hardware collaborative memory disaggregation for billion-scale approximate nearest neighbor search,” in *USENIX Annual Technical Conference (ATC)*, 2023.
- [27] S. Kim *et al.*, “AIM: Energy-efficient aggregation inside the memory for large language model inference,” in *IEEE International Solid-State Circuits Conference (ISSCC)*, 2024.
- [28] M. Zhou *et al.*, “TransPIM: A memory-based acceleration via software-hardware co-design for transformer,” in *IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, 2022.
- [29] L. Wang *et al.*, “Performance of LLM inference with PIM,” in *arXiv preprint*, 2024.
- [30] G. Yu *et al.*, “Orca: A distributed serving system for transformer-based generative models,” in *USENIX Symposium on Operating Systems Design and Implementation (OSDI)*, 2022.
- [31] Y. Sheng *et al.*, “FlexGen: High-throughput generative inference of large language models with a single GPU,” in *International Conference on Machine Learning (ICML)*, 2023.
- [32] V. A. Korthikanti *et al.*, “Reducing activation recomputation in large transformer models,” in *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, 2023.